

Actividad de recuperación día 10



AUTENTICACIÓN: USUARIOS, HASH, CSRF, MENSAJE GENÉRICO Y BLOQUEO

Fecha	Miércoles 30 de septiembre de 2026
Aprendiz	Sneider Alvarez Santos
Ficha	3428392
Documento	1016717046
Programa	Desarrollo web — SENA
Proyecto	La Carambola Dorada — sistema de gestión de salones de billar
Entorno	XAMPP — Apache 2.4.58 en el puerto 80, PHP 8.1.25, MariaDB 10.4.32
Publicación	Servidor local XAMPP — http://localhost/plan-mejoramiento-desarrollo-web/

Bitácora del día: qué hice

- 1 Las dos tablas nuevas del punto 1 en `sql/usuarios.sql`: `usuarios` con sus nueve columnas e índice único de correo, e `intentos_acceso` con una fila por cada intento de entrar, salga bien o mal.
- 2 El alta de usuarios por formulario en `registro.php`. La clave entra al POST, se convierte con `password_hash()` y a la base solo va su hash, que empieza por `$2y$`. A la columna `caracteres` solo va el número, nunca la clave.
- 3 El token CSRF del punto 3 en `app/seguridad/csrf.php`: 32 bytes al azar por sesión, comparados con `hash_equals()`, y rotados al entrar.
- 4 El mensaje genérico y el bloqueo del punto 4: siempre `Correo o contraseña incorrectos.`, y la cuenta bloqueada quince minutos al quinto fallo dentro de una ventana de quince minutos.
- 5 El ingreso que funciona y el `<script>` del punto 5: la sesión se abre, el nombre sale de la base, y un nombre con `<script>` se ve escrito y no se ejecuta.

Además quedaron por el camino dos cosas que el enunciado no pedía pero que hacen falta para que esto sea de verdad: la cookie de sesión con `httponly` y `samesite`, que en el `php.ini` de este XAMPP vienen vacíos, y un hash de mentira para que el ingreso de un correo que no existe tarde lo mismo que el de uno que sí existe.

Archivos que creé y archivos que toqué

ARCHIVO	ESTADO	QUÉ HACE
sql/usuarios.sql	Nuevo	Crea <code>usuarios</code> e <code>intentos_acceso</code> . Va aparte de <code>sql/estructura.sql</code> a propósito, para que recargar el catálogo del día 9 no borre los usuarios.
app/seguridad/sesion.php	Nuevo	<code>iniciarSesion()</code> , <code>abrirSesion()</code> , <code>cerrarSesion()</code> , <code>usuarioActual()</code> y <code>haySesion()</code> . Fija la cookie con <code>httponly</code> y <code>samesite</code> , y regenera el id al entrar.
app/seguridad/csrf.php	Nuevo	<code>tokenCsrf()</code> , <code>validarCsrf()</code> y <code>rotarCsrf()</code> . Aquí vive la constante <code>HTTP_TOKEN_INVALIDO</code> .
app/seguridad/salida.php	Nuevo	<code>esc()</code> , el <code>htmlspecialchars()</code> con el que se imprime todo dato que venga de la base o del POST.
app/controladores/Autenticacion.php	Nuevo	Las nueve funciones del día: <code>crearUsuario()</code> , <code>usuarioPorCorreo()</code> , <code>correoExiste()</code> , <code>registrarIntento()</code> , <code>fallosEnVentana()</code> , <code>estaBloqueado()</code> , <code>aplicarBloqueo()</code> , <code>levantarBloqueo()</code> y <code>verificarClave()</code> .
app/controladores/registro.php	Nuevo	<code>procesarRegistro()</code> : valida correo, rol y clave de ocho caracteres, y llama a <code>crearUsuario()</code> .
app/controladores/login.php	Nuevo	<code>procesarLogin()</code> y el mensaje genérico. Aquí está la decisión del punto 4.
registro.php	Nuevo	El formulario del punto 2. Muestra el hash que quedó en la base, que es el del punto 4.
login.php	Nuevo	El ingreso del punto 5. Al entrar bien muestra un panel de bienvenida en vez de redirigir a un tablero que aún no existe.
app/config/conexion.php	Modificado	Una línea al principio: <code>date_default_timezone_set('America/Bogota')</code> . El problema está en los problemas.
css/estilos.css	Modificado	<code>.pantalla-ingreso .sub</code> , para el texto auxiliar de los dos formularios nuevos.
dashboard.html, componentes.html, productos.php	Modificado	Solo el enlace del menú: <code>login.html</code> pasa a ser <code>login.php</code> , y la etiqueta «Cerrar sesión» pasa a ser «Ingreso».
app/menu.js	Modificado	Un comentario que mencionaba <code>login.html</code> . El código no se tocó.
login.html	Borrado	Lo reemplaza <code>login.php</code> . Un <code>.html</code> lo entrega Apache sin ejecutar nada, y aquí hace falta PHP para el token y para la consulta.

Los archivos del día 9 no se tocaron, salvo el enlace del menú. Al final se volvió a correr la prueba entera del día 9 y sigue dando **61 comprobaciones y 0 fallos**.

Punto 1. Las dos tablas nuevas

`sql/usuarios.sql` va **aparte** de `sql/estructura.sql`, y no por capricho: `estructura.sql` empieza con `DROP TABLE` de las seis tablas, así que si se volviera a cargar para trabajar en el catálogo se borrarían los usuarios que se hubieran registrado y las capturas del punto 4 dejarían de cuadrar. En el archivo de este día solo se borran y se recrean las dos tablas nuevas, y el catálogo del día 9 no se toca.

La tabla `usuarios`

COLUMNA	TIPO	PARA QUÉ SIRVE
<code>id</code>	<code>INT UNSIGNED</code> autoincremental	Número de usuario.
<code>caracteres</code>	<code>SMALLINT UNSIGNED</code>	Cuántos caracteres tiene la clave. Solo el número: por eso es una columna aparte y se rellena con <code>strlen()</code> de lo que el usuario escribió.
<code>rol</code>	<code>ENUM('administrador', 'vendedor', 'consultor')</code>	No se puede guardar un rol inventado.
<code>nombre</code>	<code>VARCHAR(80)</code>	Es donde va el <code><script></code> del punto 5.
<code>correo</code>	<code>VARCHAR(190)</code> , <code>UNIQUE</code>	No puede haber dos usuarios con el mismo.
<code>clave_hash</code>	<code>VARCHAR(255)</code>	El hash. La clave nunca se guarda.
<code>activo</code>	<code>TINYINT(1)</code>	1 = puede entrar, 0 = dado de baja.
<code>bloqueado_hasta</code>	<code>DATETIME</code> nulo	Cuándo se levanta el bloqueo. <code>NULL</code> si no está bloqueado.
<code>creado_en</code>	<code>DATETIME</code>	Momento del registro.

Por qué `clave_hash` es `VARCHAR(255)` y no `VARCHAR(60)`: hoy `password_hash()` con `PASSWORD_DEFAULT` produce bcrypt, que ocupa 60 caracteres, pero el enunciado pide 255, y ese ancho deja sitio para el día en que el algoritmo por defecto pase a `argon2`, que es más largo. Si algún día se cambia, los hashes viejos siguen cabiendo y siguen valiendo.

Y por qué el correo es `VARCHAR(190)` y no más largo: en `utf8mb4` un índice único solo puede cubrir 191 caracteres, porque cada uno puede ocupar 4 bytes y el límite de un índice InnoDB con `utf8mb4` en esta configuración son 767 bytes. Con 190 el índice entra de sobra.

La tabla `intentos_acceso`

Una fila por cada intento de entrar, salga bien o mal: `id`, `correo`, `exito`, `ip` y `creado_en`. Es lo que permite contar los cinco fallos en quince minutos sin depender de la memoria de la página, que se pierde en cuanto se recarga.

Dos decisiones de esta tabla que conviene explicar, porque las dos parecen errores y no lo son:

- **No lleva clave foránea a `usuarios`.** A propósito. El intento fallido con un correo que no existe también se guarda, porque es justo el caso interesante: si solo se guardaran los intentos de usuarios reales, el número de filas delataría si un correo está registrado o no.

- **Tampoco lleva la clave.** Ni la fallida ni la correcta. Aquí solo queda el correo, si se logró o no, desde qué IP y cuándo. Así el histórico de intentos no es un archivo de contraseñas.

El índice `idx_intentos_correo_ventana` sobre `(correo, exito, creado_en)` es el que hace la consulta del punto 4: contar los fallos de un correo en una ventana de tiempo. Sin él, habría que traer toda la tabla y filtrarla en PHP.

Punto 2. El alta de usuarios por el formulario

`registro.php` pinta el formulario y `app/controladores/registro.php` lo decide en `procesarRegistro()`. Los tres campos del enunciado y el rol, que se elige de una lista.

CAMPO	QUÉ SE VALIDA	POR QUÉ
<code>nombre</code>	No vacío, 80 caracteres como máximo	Es el campo donde va el <code><script></code> del punto 5, así que se valida su longitud pero su contenido se guarda tal cual .
<code>correo</code>	Formato de correo y que no esté repetido	Además lo comprueba el índice único <code>uq_usuarios_correo</code> en la base, no solo el PHP.
<code>clave</code>	Ocho caracteres como mínimo	Tres capas: el formulario, el PHP y el <code>CHECK (caracteres >= 8)</code> de la base, que se cumple en el servidor aunque alguien se saltase el formulario.
<code>rol</code>	Que sea uno de los tres del <code>ENUM</code>	Con <code>const ROLES = ['administrador', 'vendedor', 'consultor']</code> y el <code>ENUM</code> de la tabla.

La línea que más importa: qué se guarda y qué no

`crearUsuario()` recibe el nombre, el correo, el rol y la clave, y guarda **el hash, no la clave**:

```
$hash = password_hash($clave, PASSWORD_DEFAULT); // 60 caracteres, empieza por $2y$
$caract = strlen($clave); // a la columna
caracteres: solo el numero
// a la tabla usuarios van $nombre, $correo, $rol, $caract y $hash. La clave no entra en
ningun sitio.
```

`PASSWORD_DEFAULT` hoy es `bcrypt`, y `bcrypt` es lento a propósito: por eso un atacante con `md5` prueba millones de claves por segundo y con `bcrypt` no puede. La clave en claro no se escribe en ningún archivo: ni en el `.sql`, ni en la bitácora, ni en las capturas.

El nombre se guarda sin escapar, a propósito. Escribirlo con `htmlspecialchars()` antes de guardarlo estaría **mal**: el dato en la base tiene que ser el nombre de verdad, con su `<script>` si lo tiene. Quien lo escapa es la página, al imprimirlo, con `esc()`. Esa separación es la que hace que el punto 5 sea una prueba de verdad y no un detalle de adorno: si se escapara al guardar, la página no tendría nada que escapar al mostrar.

Por qué `usuarios.sql` no mete ni un usuario

Ninguno. Los tres usuarios del punto 2 y el del punto 5 se crean entrando al formulario, no escribiendo en el archivo. Si un usuario se escribiera ahí, su clave quedaría escrita en un `.sql` de texto plano, que es justo lo que el enunciado prohíbe. La única forma de tener usuarios es pasar por el formulario, que es donde la clave se convierte en hash.

Punto 3. El token CSRF es obligatorio

Un formulario que se manda por `POST` lo puede enviar *cualquier* página. Si el usuario tiene el formulario de ingreso abierto y sin querer entra a una página que trae esto:

```
<form action="http://localhost/.../login.php" method="post">
  <input name="correo" value="admin@carambola.test">
  <input name="clave" value="una-clave-que-ya-se">
</form>
```

el navegador manda ese formulario solo, con las cookies de la sesión, y la página queda con la sesión iniciada. El usuario no hizo nada, pero quedó dentro. Eso es un CSRF.

La defensa es que cada formulario lleve un token que el navegador no puede inventar, porque lo pone el servidor dentro de la página. En `app/seguridad/csrf.php`:

```
function tokenCsrf(): string // 32 bytes al azar -> 64 caracteres en hexadecimal
function validarCsrf(?string $enviado): bool
{
    return is_string($enviado)
        && !empty($_SESSION['csrf'])
        && hash_equals($_SESSION['csrf'], $enviado);
}
```

Por qué `hash_equals()` y no `==`: `==` compara el texto de izquierda a derecha y se devuelve en cuanto encuentra una diferencia, así que comparando el token a mano se puede acertar el bueno letra a letra midiendo cuánto tarda en responder. `hash_equals()` tarda lo mismo haya diferencias o no, y además avisa si se le pasan dos tipos distintos, que con `==` pasaría desapercibido.

Y al entrar se rota con `rotarCsrf()`: el token que estaba escrito en el formulario deja de servir, para que si alguien llega a leerlo no pueda reusarlo.

El código HTTP: por qué 403 y no el 419 del enunciado

El ejemplo del enunciado usa **419**. Aquí se usa **403**, y se probó por qué:

CÓDIGO PEDIDO	LO QUE DEVOLVIÓ APACHE	QUÉ SIGNIFICA
400	400	Correcto.
403	403	Correcto.
409	409	Correcto.
419	500	Apache en Windows no tiene la frase de estado del 419, no sabe escribir la línea de estado y la cambia por un 500.
422	422	Correcto.
429	429	Correcto.

Por qué importa la diferencia: un 500 por un token que falta sería mentira, diría que se cayó la página cuando la página está haciendo exactamente lo que debe. El 403 dice justo lo que pasó: *entendí lo que pediste y no te lo concedo*. El 419 queda anotado en el código para cuando el

servidor sepa ese código, y como el número vive en una sola constante, `HTTP_TOKEN_INVALIDO` en `app/seguridad/csrf.php`, cambiarlo allí lo cambia en toda la aplicación.

Punto 4. El mensaje genérico y el bloqueo

El mismo mensaje para todo

Da igual lo que haya pasado. Los cuatro casos devuelven la misma cadena, guardada en una constante para que no se pueda escribir de otra manera en dos sitios:

```
const MENSAJE_GENERICO = 'Correo o contraseña incorrectos.';
```

LO QUE PASO DE VERDAD	LO QUE VE EL VISITANTE	POR QUÉ IMPORTA
El correo no existe	Correo o contraseña incorrectos.	Si el mensaje fuera «ese correo no está registrado», la página serviría para saber qué correos hay dados de alta, que es medio camino del atacante.
La clave está mala	Correo o contraseña incorrectos.	
La cuenta está bloqueada	Correo o contraseña incorrectos.	
La cuenta está dada de baja	Correo o contraseña incorrectos.	

El hash de mentira, que es la otra mitad de este punto. Cuando el correo no existe no hay hash con qué comparar, así que `password_verify()` no se puede llamar y la respuesta sería más rápida que cuando el correo sí existe. Ese hueco de unos milisegundos acaba diciendo qué correos están registrados. Por eso, cuando el correo no existe, también se verifica la clave del visitante contra `HASH_SENUERO`, un bcrypt de mentira: no corresponde a ninguna clave, así que nunca entra, pero cuesta lo mismo.

El bloqueo: cinco fallos en quince minutos

Dos constantes, con los números del enunciado:

```
const MINUTOS_VENTANA = 15; // la ventana en la que se cuentan los fallos
const MAX_INTENTOS    = 5;  // cuantos fallos hay que meter en esa ventana
```

El conteo se hace en la base, con el índice del punto 1, y no con un contador en PHP: una página que se recarga no puede reiniciar la cuenta. Cada intento, salga bien o mal, deja una fila en `intentos_acceso`.

Al quinto fallo, `aplicarBloqueo()` escribe la fecha hasta la que queda:

```
UPDATE usuarios
  SET bloqueado_hasta = DATE_ADD(NOW(), INTERVAL 15 MINUTE)
 WHERE id = :id
```

El campo es una fecha y no un sí/no por una razón concreta: así el bloqueo **se levanta solo, sin que nadie lo toque**. En quince minutos `bloqueado_hasta` ya es un pasado y la cuenta vuelve a estar usable sin que nadie la desbloquee a mano. Si fuera un `SI/NO`, haría falta un proceso que lo apagara, y ese proceso habría que existir, ejecutarse y no romperse nunca.

En `aplicarBloqueo()` el 15 va concatenado en la cadena SQL. No es una inyección: es la constante entera `MINUTOS_VENTANA`, que no viene de fuera y no se puede cambiar. Todo lo que *sí* viene del visitante va con marcador y `bindValue()`.

Punto 5. El ingreso que funciona, y el `<script>` que no se ejecuta

Entrar

`procesarLogin()` hace, en este orden: validar el token, buscar el correo, contar los fallos de la ventana, comprobar el bloqueo, verificar la clave y abrir la sesión. Si algo falla, el mismo mensaje genérico y un intento más en la tabla.

La sesión se abre con `abrirSesion()`, y ahí hay dos detalles que no son finales:

- `session_regenerate_id(true)` **al entrar**. El id de sesión que llevaba el formulario deja de valer, de modo que si alguien se había quedado con ese id no puede reutilizarlo.
- **La cookie con `httponly` y `samesite`**. El `php.ini` de este XAMPP los deja **vacíos**, y por eso hay que ponerlos a mano en `session_set_cookie_params()`. Sin `httponly`, cualquier JavaScript de la página puede leer la cookie de sesión. `secure` queda en `false` porque esto corre en `http://localhost`, sin TLS: en un servidor de verdad sería `true`.

Al entrar bien, `login.php` **no redirige a ninguna parte**: muestra un panel de bienvenida con el nombre, el correo y el rol, que salen de la base y de la sesión, todos por `esc()`. El tablero con sesión es el día 11, y redirigir a una página que aún no existe habría sido dejar el punto 5 a medias.

El `<script>`

Se registró un usuario cuyo nombre es `<script>alert(1)</script>`, y en la base se guardó tal cual. Al entrar con ese usuario, el nombre se imprime así:

```
<p>Bienvenido, <strong id="nombre-usuario">&lt;script&gt;alert(1)&lt;/script&gt;</strong>.</p>
```

El navegador no lo ejecuta: sale escrito, con los caracteres `<` y `>` a la vista. Y el motivo de que la prueba valga es que el dato **sale de la base de datos**, no de un texto escrito a mano en la plantilla: si el nombre se escapara al guardarlo, la página no tendría nada que escapar al mostrar.

COMPROBACIÓN	RESULTADO
El navegador mostró alguna alerta	<code>0 alertas</code>
El texto <code><script></code> salió escrito en la página	<code>Sí</code>
En la base, el nombre está sin escapar	<code>Sí</code> , con el <code><script></code> dentro

Los criterios de evaluación, punto por punto

CRITERIO	VEREDICTO	CÓMO SE COMPRUEBA
Las dos tablas con las columnas pedidas	CUMPLE	<code>usuarios</code> con sus 9 columnas y <code>intentos_acceso</code> con sus 5, leídas de <code>information_schema.columns</code> . Índice único <code>uq_usuarios_correo</code> confirmado.
La clave se guarda con hash, no en claro	CUMPLE	Los cuatro usuarios tienen un hash de 60 caracteres que empieza por \$2y\$, y los cuatro son distintos entre sí , porque cada llamada lleva su propia sal. En <code>intentos_acceso</code> no hay ninguna columna de clave.
Sin <code>md5</code> ni <code>sha1</code>	CUMPLE	Un escaneo de los archivos PHP del día no encuentra ninguna llamada a <code>md5</code> , <code>sha1</code> , <code>crc32</code> ni comparaciones de clave con <code>==</code> .
El token CSRF es obligatorio	CUMPLE	Un <code>POST</code> sin token: 403 . Un <code>POST</code> con un token inventado: 403 . Sin cookie de sesión, tampoco entra ni con la clave buena: 403 . Y el token de un formulario en blanco mide 64 caracteres .
El mismo mensaje en los cuatro casos	CUMPLE	Correo inexistente, clave mala, cuenta bloqueada y cuenta dada de baja devuelven los cuatro exactamente la misma cadena. En el código solo hay una constante con ese texto.
Bloqueo a los cinco intentos	CUMPLE	Tras cinco fallos seguidos quedan 5 filas en <code>intentos_acceso</code> y <code>bloqueado_hasta</code> con fecha de futuro. Con la clave correcta tampoco entra, y el mensaje sigue siendo el genérico. Pasada la fecha, vuelve a entrar solo.
El <code><script></code> no se ejecuta	CUMPLE	Cero alertas en el navegador y el texto escrito en la página, con el dato viniendo de la base.
No hay ni una clave en claro en el proyecto	CUMPLE	Ni en el <code>.sql</code> , ni en las capturas, ni en el código. Los usuarios se crean por el formulario, que es donde la clave se hashea.

Los problemas que me salieron y cómo los resolví

El bloqueo se caía solo, y el culpable no era el código del bloqueo

El síntoma: se bloquea la cuenta al quinto fallo, y al momento de probarlo con la clave correcta la cuenta ya entraba. El bloqueo de quince minutos parecía expirado nada más empezar. Un bloqueo que no bloquea no sirve de nada, así que tocó revisar.

La causa no estaba en `aplicarBloqueo()`, que escribía la fecha bien. Estaba en cómo se leía esa fecha: `estaBloqueado()` la pasaba por `strtotime()` y la comparaba con `time()`, las dos en PHP. Pero la fecha la había escrito MySQL, y en esta máquina **PHP corre en Europe/Berlin y MySQL en America/Bogota: siete horas distintas**. O sea, se comparaba una hora de Bogotá contra una hora de Berlín.

Se resolvió por dos lados, porque las dos cosas estaban mal:

- **La pregunta la hace la base, no PHP.** La columna que se compara ya no se trae y se procesa: se calcula en la propia consulta, con el `NOW()` del propio MySQL, que es la misma hora con la que la escribió:

```
(bloqueado_hasta IS NOT NULL AND bloqueado_hasta > NOW()) AS bloqueado
```

Así las dos horas ya no tienen por qué coincidir. Además se añadió

`date_default_timezone_set('America/Bogota')` en `app/config/conexion.php`, para que la aplicación no quede ajena al problema.

Se comprobó con tres casos, que son los que importan: bloqueado, da `bloqueado = 1` y `estaBloqueado() = true` aunque `password_verify()` siga diciendo que la clave es correcta; bloqueado ya caducado, da `false`; y `bloqueado_hasta` en `NULL`, también `false`.

El hash que se mostraba no era el que quedó guardado

El registro tiene que enseñar el hash que se va a comprobar en el punto 4, así que la página lo muestra. La primera versión pasaba la clave otra vez por `password_hash()` para tener algo que pintar. El hash de la pantalla **no era el de la base**: cada llamada a `password_hash()` le pone una sal nueva, así que salía un hash distinto cada vez, y el punto 4 habría estado comparándose con una cosa que no estaba en la tabla.

Se resolvió haciendo que `crearUsuario()` devuelva el hash que guardó, y no uno nuevo:

```
@return array{id: int, clave_hash: string} // el hash guardado, no uno recién hecho
```

Y se añadió a la prueba una comparación de que el hash de la pantalla y el de la base **son el mismo texto**, que es justo el fallo que se habría colado.

Un falso positivo en la regresión del día 9

Al borrar `login.html` y dejar `login.php` en su lugar, la prueba del día 9 seguía midiendo `login.html`. Seguí dando «no desborda» en los cinco anchos, y el resultado parecía perfecto. **Era mentira**: la página que se estaba midiendo era el 404 de Apache, y un 404 no desborda nunca. Una comprobación que se pasa porque no estaba mirando lo que cree que mira es peor que no tenerla, porque ocupa el sitio de una de verdad.

Se corrigieron las dos cosas: la prueba ahora mide `login.php`, y añadió una comprobación de que la página entregó de verdad el formulario y no un 404, mirando que estén el `h1`, el `form` y el token `csrf` con sus 64 caracteres. El día 9 pasó de 60 a **61 comprobaciones**.

El 419 que salía convertido en 500

Contado en el punto 3. Se probó con una sonda que pedía 400, 403, 409, 419, 422 y 429: todos pasaban tal cual menos el 419, que Apache en Windows no sabe escribir y cambiaba por un 500. Se eligió el 403 y quedó escrito por qué en el propio archivo, con la sonda y su resultado, para que quien lea no piense que el 403 es un descuido.

Cómo se verificó

COMPROBACIÓN	CÓMO	RESULTADO
Sintaxis de los PHP que toca el día	<code>php -l</code> sobre cada uno	Los 9 sin errores (8 nuevos y <code>conexion.php</code>)
Carga de <code>usuarios.sql</code>	<code>mysql -u root < sql/usuarios.sql</code>	Sin un solo error
Columnas de las dos tablas	<code>information_schema.columns</code>	9 en <code>usuarios</code> , 5 en <code>intentos_acceso</code>
Correo único	<code>information_schema.statistics</code>	<code>uq_usuarios_correo</code> declarado
Los cuatro usuarios del formulario	Alta por <code>registro.php</code>	1 administrador, 1 vendedor, 2 consultor
Los cuatro hashes	Leídos de la base	60 caracteres, empiezan por \$2y\$ y son los cuatro distintos
Que no haya ninguna clave en claro	Buscando la clave de la prueba en la base, el <code>.sql</code> y las capturas	Cero apariciones
El token CSRF	Tres POST reales: sin token, con token falso, sin cookie de sesión	Los tres, 403
El mensaje genérico	Correo inexistente, clave mala, bloqueada y dada de baja	Los cuatro, el mismo texto
El bloqueo	Cinco fallos seguidos y luego la clave correcta	5 filas en <code>intentos_acceso</code> , y con la clave buena tampoco entra
Que el bloqueo se levante solo	Bloqueo caducado y <code>NULL</code>	Los dos, false
El <code><script></code>	Contando las alertas del navegador	0 alertas , y el texto escrito en la página
Lo que el enunciado prohíbe	Buscando <code>md5</code> , <code>sha1</code> , <code>crc32</code> y claves en claro	Cero apariciones
Los enlaces viejos	Buscando <code>login.html</code> en los archivos que se sirven	Cero : ya no queda ninguno
Desborde horizontal	320, 360, 768, 1024 y 1440 px	Ninguna de las dos páginas nuevas desborda
Que las capturas no salgan en blanco	Contando colores distintos en cada PNG	Las cuatro con contenido : 180, 182, 199 y 1724 colores
Regresión del día 9	La prueba entera del día 9, con la corrección del 404	61 comprobaciones, 0 fallos
El HTML de esta bitácora	Validador W3C Nu	0 errores, 0 avisos

COMPROBACIÓN	CÓMO	RESULTADO
El ancho de esta bitácora	360, 768, 1024 y 1440 px	Ninguna tabla se sale del ancho útil
Total del día 10	Todo lo anterior, en Edge real	79 comprobaciones, 0 fallos

Evidencia

**LA CARAMBOLA
DORADA**
SISTEMA DE SALONES DE BILLAR

Ingreso al panel de gestión

Sistema de gestión de salones de billar

Sesión iniciada

Bienvenido, **Ana Rivera**.

- Correo: admin@caramboladorada.co
- Rol: **administrador**
- Entró a las: 2026-09-27 11:10:33

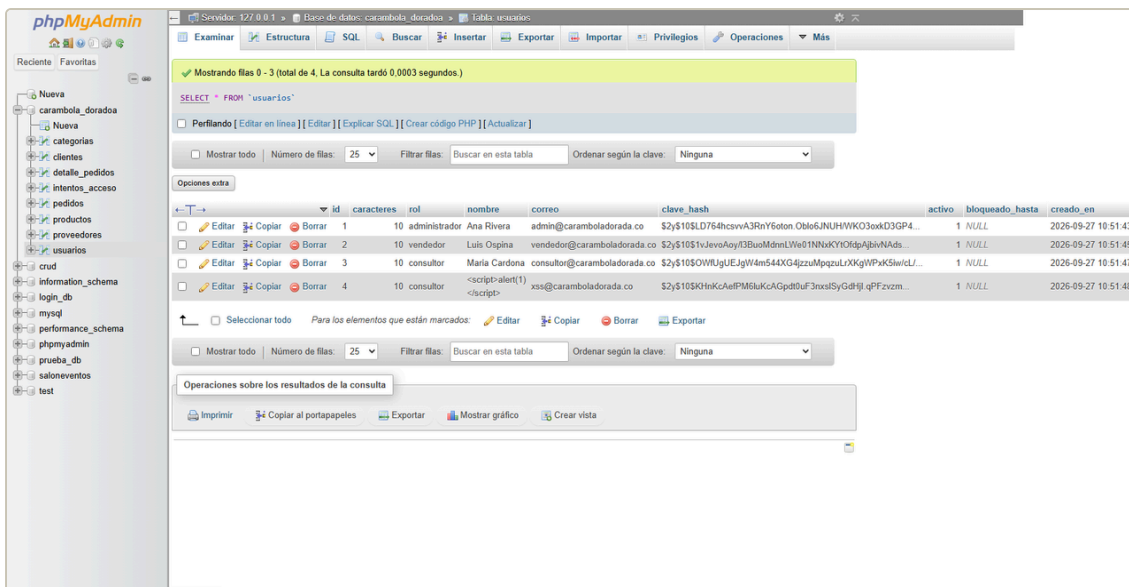
El nombre de arriba sale de la base de datos y se imprime con `htmlspecialchars()`. Si ese nombre trae un `<script>`, se ve escrito y no se ejecuta.

[Registrar otro usuario](#)

Este mensaje de error es el mismo para un correo que no está registrado, para una contraseña equivocada y para una cuenta bloqueada: si fueran distintos, la página serviría para averiguar qué correos existen.

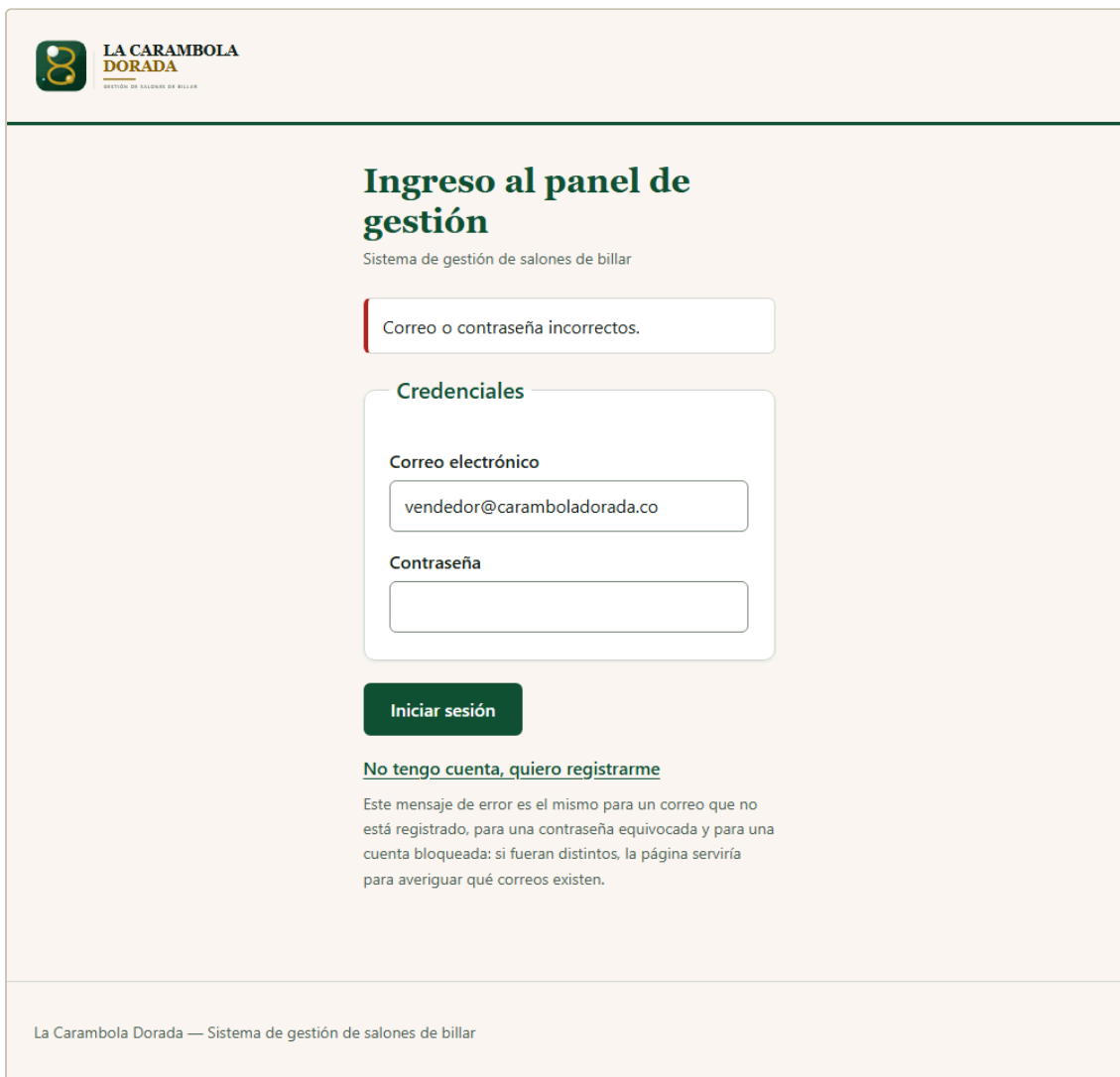
La Carambola Dorada — Sistema de gestión de salones de billar

Punto 5. El ingreso que funciona. El nombre, el correo y el rol salen de la base de datos y pasan por la sesión, y los tres se imprimen con `esc()`. La página no redirige a ninguna parte: el tablero con sesión es el día 11.



	id	caracteres	rol	nombre	correo	clave_hash	activo	bloqueado_hasta	creado_en
☐	1	10	administrador	Ana Rivera	admin@caramboladorada.co	\$2y\$10\$LD764hcsrvA3RnY6oton.Oblo6JNUHWWK03oxD3GP4...	1	NULL	2026-09-27 10:51:43
☐	2	10	vendedor	Luis Ospina	vendedor@caramboladorada.co	\$2y\$10\$5tvJevuAoyfI3BuoMdmLW601NNxKY0f6pAjbvNAds...	1	NULL	2026-09-27 10:51:45
☐	3	10	consultor	Maria Cardona	consultor@caramboladorada.co	\$2y\$10\$OWlUgUEJgW4m544XG4jzzuMpqzLXXgWPxK5wlcL...	1	NULL	2026-09-27 10:51:47
☐	4	10	consultor	<script>alert(1)</script>	xxs@caramboladorada.co	\$2y\$10\$KHnKcAaIPM5uKcAGpdtuf3ncuSyGd8fj qPFzvmz...	1	NULL	2026-09-27 10:51:48

Punto 2. Los cuatro hashes en `phpMyAdmin`. Los cuatro empiezan por `$2y$`, miden 60 caracteres y **son distintos entre sí**, porque cada registro lleva su propia sal. Ni una clave en claro, ni en la tabla ni en la de intentos.



**LA CARAMBOLA DORADA**
SISTEMA DE SALONES DE BILLAR

Ingreso al panel de gestión

Sistema de gestión de salones de billar

Correo o contraseña incorrectos.

Credenciales

Correo electrónico

vendedor@caramboladorada.co

Contraseña

Iniciar sesión

[No tengo cuenta, quiero registrarme](#)

Este mensaje de error es el mismo para un correo que no está registrado, para una contraseña equivocada y para una cuenta bloqueada: si fueran distintos, la página serviría para averiguar qué correos existen.

La Carambola Dorada — Sistema de gestión de salones de billar

Punto 4. La cuenta bloqueada tras cinco intentos. Lo que se ve es el mensaje genérico, y es lo mismo que se vería con la clave correcta: no se dice si el correo existe, ni si está bloqueado, ni si la clave estaba mal. Esa es la gracia del mensaje genérico, y es lo que la captura demuestra.



**LA CARAMBOLA
DORADA**
SISTEMA DE SALONES DE BILLAR

Ingreso al panel de gestión

Sistema de gestión de salones de billar

Sesión iniciada

Bienvenido, `<script>alert(1)</script>`.

- Correo: xss@caramboladorada.co
- Rol: **consultor**
- Entró a las: 2026-09-27 11:10:31

El nombre de arriba sale de la base de datos y se imprime con `htmlspecialchars()`. Si ese nombre trae un `<script>`, se ve escrito y no se ejecuta.

[Registrar otro usuario](#)

Este mensaje de error es el mismo para un correo que no está registrado, para una contraseña equivocada y para una cuenta bloqueada: si fueran distintos, la página serviría para averiguar qué correos existen.

La Carambola Dorada — Sistema de gestión de salones de billar

Punto 5. El nombre con `<script>` sale escrito, con las etiquetas a la vista, y el navegador no ejecutó nada: cero alertas. El dato viene de la base, y la base lo guarda sin escapar a propósito; lo escapa la página al imprimirlo.

Cómo se tomaron estas capturas, para que quede claro: se abrió Edge con las medidas reales, se mandó el formulario de verdad, y antes de cada foto se comprobó el estado de la página en ese instante: los cuatro hashes con su `$2y$`, el texto literal del ataque en el nombre, el mensaje genérico en pantalla, y el contador de alertas del navegador a cero. Las capturas son del navegador; no las dibujé yo. La de `phpMyAdmin` va reescalada a 1280 px de ancho para que no se salga de la hoja.

Lo que no hice, y por qué

- **La página de cerrar sesión.** Por decisión tuya, y no por el enunciado: el menú ahora dice «Ingreso» y apunta a `login.php`. El `cerrarSesion()` está escrito y probado, pero la página que lo llama es del día 11, con el tablero de verdad.
- **Usar 419 en vez de 403.** No fue elección estética: en este Apache el 419 sale como 500. Contá en el punto 3 y está en el código, con la sonda, para que conste.
- **Recuperar la clave.** Pedir un correo nuevo y generar un enlace es un sistema entero, con su tabla y sus correos, y no lo pide este día.
- **Un segundo factor o una sesión corta.** Tampoco lo pide. La cookie va con `httponly` y `samesite`, que es lo que el día sí permite hacer sin inventar.
- **Un usuario de MySQL propio para la aplicación.** Se conecta con `root` sin clave, que es el default de XAMPP y sirve para clase. En un servidor de verdad haría falta un usuario con permisos solo sobre esta base.

- **Un repositorio git.** El `.gitignore` está escrito y con la regla correcta, pero el proyecto todavía no es un repositorio.

Sneider Alvarez Santos
Aprendiz — Ficha 3428392

Recepción / Instructor

La Carambola Dorada — Actividad de recuperación día 10

Sneider Alvarez Santos · 1016717046 · ficha 3428392